UNIT II – FUNCTIONS & BOOLEAN ALGEBRA

Functions: Definition, Classification of functions, Operations on functions. Growth of Functions.
Boolean Algebra: Introduction, Axioms and Theorems of Boolean algebra, Algebraic manipulation of Boolean expressions. Simplification of Boolean Functions, Karnaugh maps.

◇ 1. Functions

🧩 Definition

A **function** is a mathematical relation that gives **one output for every input**.
In digital logic, functions describe the **relationship between input variables and output variables** in a logic circuit.

💡 **Example:**
Let (f(A, B) = A + B')
This means:

- The function (f) has two inputs: A and B.
- The output is **1** when A = 1 **or** B = 0.

So, this function can be implemented using a **NOT gate** and an **OR gate**.

◇ Classification of Functions

Type of Function	Description	Example
One-to-One Function	Each input maps to a unique output	(f(x) = x + 1)
Many-to-One Function	Two or more inputs can produce same output	(f(x) = x^2): both 2 and -2 give 4
Onto Function	Every possible output has a corresponding input	Logical functions where all output values (0 or 1) occur
Into Function	Some output values have no corresponding input	Function giving only 0 outputs
Constant Function	Output is same for all inputs	(f(x) = 1) or (f(x) = 0)

🔹 In digital logic, most Boolean functions are **many-to-one**.

◇ Operations on Functions

We can perform the following operations on Boolean functions:

Operation	Symbol	Example	Description
AND	(.)	(f = A.B)	Output 1 only if both inputs are 1
OR	(+)	(f = A + B)	Output 1 if any input is 1
NOT	(')	(f = A')	Output is complement (opposite) of input
XOR (Exclusive OR)	$\oplus$	(f = A $\oplus$ B)	Output 1 if inputs are different
XNOR (Exclusive NOR)	$\oplus$	(f = (A $\oplus$ B)')	Output 1 if inputs are same

💡 **Example:**

If (f(A, B) = A.B'),

Then:

- (f(0, 0) = 0)
- (f(0, 1) = 0)
- (f(1, 0) = 1)
- (f(1, 1) = 0)

❁ 2. Growth of Functions

This topic often means **how output grows or changes as input variables increase**.

In Boolean logic:

- Number of possible input combinations = (2n), where (n) = number of input variables.
- Number of possible Boolean functions = (2^{(2^n)})

💡 **Example:**

For 2 input variables (A, B):

- Input combinations = (2^2 = 4)
- Possible Boolean functions = (2^{(2^2)} = 16)

◇ 3. Boolean Algebra

Introduction

Boolean Algebra is a **mathematical system** used to **analyze and simplify digital logic circuits**.

It deals with only two values:

- **1 → TRUE / HIGH**
- **0 → FALSE / LOW**

Invented by **George Boole (in 1854)**.
Used in:

- Simplifying logic expressions
- Designing combinational circuits
- Representing logic gates algebraically

◇ Basic Boolean Operations

Operation	Symbol	Expression	Equivalent Gate
AND	.	(A .B)	AND gate
OR	(+)	(A + B)	OR gate
NOT	(')	(A')	NOT gate

◇ 4. Axioms and Theorems of Boolean Algebra

🧩 Axioms (Basic Laws)

Law	Expression	Example
Identity Law	(A + 0 = A), (A.1 = A)	(A + 0 = A)
Null Law	(A + 1 = 1), (A . 0 = 0)	(A .0 = 0)
Idempotent Law	(A + A = A), (A . A = A)	(A + A = A)
Complement Law	(A + A' = 1), (A .A' = 0)	(A .A' = 0)
Commutative Law	(A + B = B + A), (A . B = B .A)	
Associative Law	((A + B) + C = A + (B + C))	
Distributive Law	(A .(B + C) = A .B + A .C)	

🧮 Important Theorems

Theorem	Expression	Meaning
Absorption	(A + A \cdot B = A)	A absorbs AB
De Morgan's	((A .B)' = A' + B') and ((A + B)' = A' .B')	Convert AND↔OR with NOT
Involution Law	((A')' = A)	Double complement returns same value
Consensus Theorem	(AB + A'C + BC = AB + A'C)	Redundant term removal

💡 **Example Using Theorems:**

Simplify: (Y = A + A'B)
→ (Y = (A + A')(A + B)) (using distributive)
→ (Y = 1 .(A + B))
→ (Y = A + B)

◇ 5. Simplification of Boolean Functions

Simplifying Boolean functions reduces the number of logic gates.

Methods:

1. **Algebraic Simplification** – using laws & theorems manually.
2. **Karnaugh Map (K-map)** – graphical method.

◇ 6. Karnaugh Maps (K-Map)

Introduction

K-map is a **pictorial method** to simplify Boolean expressions.

It eliminates the need for long algebraic manipulations.

No. of Variables	Size of K-Map
2	2×2 = 4 cells
3	2×3 = 8 cells
4	4×4 = 16 cells

Example: 3-variable K-map

Variables: A, B, C

Number of cells = (2^3 = 8)

AB\C	0	1
00	m0	m1
01	m2	m3
11	m6	m7
10	m4	m5

Steps to Simplify Using K-map

1. Write the Boolean function in **sum of minterms (SOP)** form.
2. Plot 1's in K-map where function = 1.
3. Group adjacent 1's in powers of 2 (1, 2, 4, 8...).
4. Write simplified expression for each group.
5. Combine all groups.

Example:

Given (F(A, B, C) = Σ (1, 3, 5, 7))

AB\C	0	1
00	0	1
01	0	1

11	0	1
10	0	1

→ Groups of 4 adjacent 1s form one group.

Simplified Expression:

(F = B' + C)

Advantages of K-map

- Fast and visual simplification
- Reduces human error
- Gives **minimum number of gates**

Important Difference Tables – Unit II (Functions & Boolean Algebra)

1. Boolean Function vs Arithmetic Function

Basis	Boolean Function	Arithmetic Function
Domain	Binary values (0, 1)	Real or integer numbers
Operations Used	Logical (AND, OR, NOT)	Arithmetic (+, −, ×, ÷)
Output Values	0 or 1 only	Any numeric value
Application	Logic circuits, digital design	Mathematical calculations
Example	(f(A,B) = A + B')	(f(x,y) = x + y)

2. Boolean Algebra vs Ordinary Algebra

Basis	Boolean Algebra	Ordinary Algebra
Variables	Binary (0 or 1)	Real numbers
Basic Operators	AND (·), OR (+), NOT (')	+, −, ×, ÷
Laws	Idempotent, Complement, De Morgan's	Associative, Commutative, Distributive
Values of Variable	Only 0 or 1	Infinite values
Purpose	Simplify logic circuits	Solve numerical problems
Example	(A + A'B = A + B)	(x + xy = x(1 + y))

🔢 3. Logic Operation: AND vs OR vs NOT

Feature	AND (·)	OR (+)	NOT (')
Meaning	Multiplication / all true	Addition / any true	Inversion
Output	1 if all inputs = 1	1 if any input = 1	Opposite of input
Symbol	·	+	'
Gate Used	AND Gate	OR Gate	NOT Gate
Example	(A·B)	(A+B)	(A')

🔁 4. Sum of Products (SOP) vs Product of Sums (POS)

Basis	SOP (Sum of Products)	POS (Product of Sums)
Form	OR of AND terms	AND of OR terms
Basic Operation	Minterm (1-output form)	Maxterm (0-output form)
When Used	When output = 1 for minterms	When output = 0 for maxterms
Simplified Form	(F = A'B + AB')	(F = (A + B)(A' + C))
Circuit Type	AND–OR logic	OR–AND logic
K-map Representation	Fill 1s for minterms	Fill 0s for maxterms

🧮 5. Minterm vs Maxterm

Basis	Minterm	Maxterm
Definition	Product (AND) of all variables in true/complement form giving output 1	Sum (OR) of all variables giving output 0
Symbol	(m_i)	(M_i)
Logical Form	AND form (e.g., (A'BC))	OR form (e.g., (A + B' + C'))
Used In	SOP form	POS form
Output Condition	Function = 1	Function = 0
K-map Marking	1	0

6. Algebraic Simplification vs K-map Simplification

Basis	Algebraic Method	K-map Method
Type	Mathematical manipulation	Graphical method
Difficulty Level	Tedious for >3 variables	Simple up to 4–6 variables
Accuracy	Prone to human error	Visual and accurate
Representation	Uses Boolean laws	Uses cells & grouping
Best For	Theoretical proofs	Circuit design simplification

7. De Morgan's Law (Two Forms)

Form	Expression	Meaning
First Law	((A·B)' = A' + B')	Complement of AND → OR of complements
Second Law	((A + B)' = A'·B')	Complement of OR → AND of complements

8. Half Adder vs Full Adder

Basis	Half Adder	Full Adder
Inputs	2 (A, B)	3 (A, B, Cin)
Outputs	2 (Sum, Carry)	2 (Sum, Carry)
Carry Input	Not available	Available
Expression for Sum	(S = A ⊕ B)	(S = A ⊕ B ⊕ Cin)
Expression for Carry	(C = A·B)	(C = AB + BCin + ACin)
Use	Basic addition	Multi-bit addition

9. Logic 0 vs Logic 1

Basis	Logic 0	Logic 1
Voltage Level	Low voltage (0V or near)	High voltage (e.g., +5V)
Boolean Meaning	FALSE	TRUE
Lamp/LED Output	OFF	ON
Binary Representation	0	1

10. Function vs Expression

Basis	Function	Expression
Definition	Relation showing output for inputs	Formula representing relationship
Example	(f(A,B) = A + B')	(A + B')
Representation	Symbolic mapping (f)	Algebraic form
Use	Logical operation representation	Simplification and analysis